

สรุป Preprocessing และ Hyperparameters

AI Classification of Cyber Attacks — CICIDS2017 (2,497,980 แถว, 9 คลาส, temporal split 70/30)

อ้างอิงจากโค้ด `train.py` และผลลัพธ์ `results/cicids2017_temporal_v1/` · จัดทำ 28 ก.ย. 2026

1. Preprocessing

1.1 การทำความสะอาดข้อมูล (ทำที่ไฟล์ CSV)

ที่มา: `train.py` ฟังก์ชัน `_clean_one_frame`

- ตัดช่องว่างหน้า/หลังชื่อคอลัมน์ (ไฟล์ CICIDS2017 มีช่องว่างนำหน้าชื่อ)
- บันทึก `_row_index` (ตำแหน่งแถวในไฟล์ CSV ดิบ) ก่อนลบแถวใด ๆ เพื่อใช้เป็นตัวเรียงลำดับเวลาของ split
- เปลี่ยนชื่อ `Fwd Pkts/s` → `Fwd Packets/s` และ `Bwd Pkts/s` → `Bwd Packets/s`
- ลบคอลัมน์ซ้ำ `Fwd Header Length.1`
- ลบคอลัมน์ที่ทำให้ข้อมูลซ้ำ (ถ้ามี): `Flow ID`, `Source/Destination IP`, `Src/Dst IP`, `Source Port`, `Src Port`, `Timestamp`
- ลบคอลัมน์ `Protocol`
- ลบแถวที่เป็น header เสีย (ค่า label เท่ากับ `"Label"`)
- แปลง label ด้วย `normalize_label` จาก 15 label ดิบ เหลือ 9 คลาส: `BENIGN`, `DoS`, `DDoS`, `PortScan`, `Bot`, `Web Attack`, `Brute Force`, `Infiltration`, `Heartbleed` — แถวที่แปลงได้ `Other` จะถูกลบ
- บังคับทุก feature เป็นตัวเลข (`pd.to_numeric(errors="coerce")`) แล้วแปลงเป็น `float32`
- แปลงค่า `±Inf` เป็น `NaN` แล้วลบแถวที่มี `NaN`

1.2 หลังรวมทุกไฟล์

ที่มา: `load_and_clean_cached`

- ตรวจว่ามีครบ 8 capture file — ถ้าขาดจะหยุดทำงาน เพราะแต่ละคลาสอยู่ในไฟล์เดียว ขาดไฟล์คือคลาสหายทั้งคลาส
- เพิ่ม metadata `source_file`, `dataset_id`, `capture_window` (ไม่ป้อนเข้าโมเดล)
- ลบแถวที่ซ้ำกันทุกคอลัมน์ข้ามไฟล์ (ไม่นับคอลัมน์ metadata)
- cache ไว้ที่ `data/processed/cicids2017_clean.parquet` (2,497,980 แถว)

1.3 การแบ่ง Train / Test

- ตรวจลำดับเวลาของ `_row_index` เทียบกับตารางโจมตีที่ CIC เผยแพร่ (`validate_capture_chronology`)
- แบ่งตามเวลา **70/30** ภายในแต่ละกลุ่ม (ไฟล์, คลาส) (`temporal_source_split`) โดยทุกคลาสมีอย่างน้อย 3 แถวใน test (`min_test_per_class=3`)
- ตรวจจำนวนแถวต่อคลาสเทียบกับ manifest `configs/splits/cicids2017_temporal_70_30.json` — ไม่ตรงจะหยุดทำงาน
- ไม่มีชุด calibration (`threshold_validation_size = 0.0`)
- `LabelEncoder` เข้ารหัส label หลังแบ่งข้อมูลแล้ว

1.4 Pipeline ภายในโมเดล

ทุกโมเดลใช้โครงสร้างเดียวกัน และ fit ใหม่ทุก CV fold จึงไม่มีข้อมูลรั่วจาก test/validation

```
SimpleImputer(strategy="median") → StandardScaler() → [sampler ถ้าเลือก] → classifier
```

1.5 การจัดการคลาสไม่สมดุล

ค่าเริ่มต้น `imbalance_strategy = "class_weight"` ใช้กับทุกโมเดลเหมือนกัน:

โมเดล	วิธีถ่วงน้ำหนัก
Random Forest	<code>class_weight="balanced_subsample"</code>
LightGBM, Logistic Regression, meta-learner ของ Stacking	<code>class_weight="balanced"</code>
CatBoost	<code>auto_class_weights="Balanced"</code>
XGBoost, MLP	ไม่มีพารามิเตอร์นี้ จึงใช้ wrapper <code>BalancedXGBClassifier</code> / <code>BalancedMLPClassifier</code> ที่คำนวณ <code>compute_sample_weight("balanced")</code> ภายในแต่ละ fold

ทางเลือกอื่น: `targeted`, `random_over`, `borderline_smote`, `smoteenn` (`k_neighbors=5`) — รันภายใน pipeline

ข้อสังเกต: `subsample_n` ใน CONFIG และใน preset ถูกอ่านไปใช้แค่ในโหมด `--smoke` และในรายงานเท่านั้น ในเส้นทาง temporal ข้อมูลที่ใช้คือทั้ง corpus หลังทำความสะอาด โดยไม่มีการ subsample

2. การค้นหา Hyperparameter

รายการ	ค่า
วิธี	<code>RandomizedSearchCV</code> , <code>refit=False</code> แล้วนำ best params ไป fit ใหม่บน train ทั้งหมด
Objective	<code>f1_macro</code> (ให้น้ำหนักทุกคลาสโจมตีเท่ากัน)
CV	<code>StratifiedKFold(n_splits=5, shuffle=True, random_state=42)</code>
ขนาด search space	144 ชุด เท่ากันทุกโมเดล (<code>HP_SEARCH_SPACE_SIZE</code>)
Preset 32gb (ใช้ในรันนี้)	<code>n_iter=20</code> (≈13.9% ของ space), HP subset 200,000 แถว, <code>n_jobs=2</code>
HP subset	เก็บทุกแถวของคลาสที่มี ≤200 แถว ส่วนคลาสอื่นสุ่มตามสัดส่วน (<code>_hp_subset</code>)
ไม่ค้นหา	<code>class_weight</code> และ <code>max_iter</code> (ตามกฎหมายของโปรเจกต์)
Seed	<code>random_state = 42</code> ทุกที่

3. Hyperparameters ต่อโมเดล

โมเดล	ค่าคงที่ (ไม่ได้ tune)	Search grid	Best (รัน temporal_v1)
Random Forest	n_jobs=-1	n_estimators [200, 300, 400, 500] max_depth [None, 20, 30, 40] min_samples_split [2, 5, 10] min_samples_leaf [1, 2, 4]	n_estimators=300 max_depth=30 min_samples_split=2 min_samples_leaf=1
XGBoost	colsample_bytree=0.9, tree_method=hist, objective=multi:softprob, eval_metric=mlogloss, device=cuda	n_estimators [200, 400, 600, 800] max_depth [6, 8, 10] learning_rate [0.03, 0.05, 0.1, 0.15] subsample [0.8, 0.9, 1.0]	n_estimators=800 max_depth=10 learning_rate=0.05 subsample=0.8
LightGBM	feature_fraction=1.0, bagging_fraction=1.0, bagging_freq=0, min_split_gain=0, force_col_wise	n_estimators [400, 600, 800, 1000] num_leaves [63, 127, 255] learning_rate [0.03, 0.05, 0.1, 0.15] min_child_samples [1, 2, 5]	n_estimators=1000 num_leaves=127 learning_rate=0.1 min_child_samples=1
CatBoost	verbose=0, task_type=GPU	iterations [300, 400, 500, 600] depth [6, 8, 10] learning_rate [0.03, 0.05, 0.1, 0.15] l2_leaf_reg [1, 3, 5]	iterations=500 depth=6 learning_rate=0.15 l2_leaf_reg=1
MLP	activation=relu, solver=adam, max_iter=500, early_stopping (validation 0.1, patience 20)	hidden_layer_sizes [(128,64), (256,128,64), (256,128,64,32), (512,256,128)] alpha [1e-5, 1e-4, 1e-3] learning_rate_init [0.0005, 0.001, 0.003, 0.005] batch_size [256, 512, 1024]	hidden=(256,128,64,32) alpha=1e-4 learning_rate_init=0.0005 batch_size=256
Logistic Regression	solver=lbfgs, max_iter=2000	C 24 ค่า (0.001 – 500) tol [1e-4, 5e-4, 1e-3] fit_intercept [True, False]	C=300 tol=1e-4 fit_intercept=True
Stacking	ไม่ tune — StackingClassifier ต้อง refit base estimator ทุกตัวต่อ 1 candidate จึง รายงานเป็น untuned แทนการให้งบค้นหาที่น้อยกว่าและไม่ยุติธรรม (HP_UNTUNED_MODELS)		—

รายละเอียด Stacking

ส่วนประกอบ	ค่า
Base: LightGBM	n_estimators=300, num_leaves=63, learning_rate=0.05, feature_fraction=0.9, bagging_fraction=0.9, bagging_freq=5
Base: XGBoost	n_estimators=200, max_depth=8, learning_rate=0.1, subsample=0.9, colsample_bytree=0.9
Base: Random Forest	n_estimators=200
Meta-learner	LogisticRegression(C=1.0, solver=lbfgs, max_iter=1000)
การรวม	cv=5, stack_method="predict_proba", passthrough=False

ค่าเริ่มต้นก่อน tune (ใน build_pipeline)

โมเดล	ค่าเริ่มต้น
Random Forest	n_estimators=400, max_depth=None, min_samples_split=2
XGBoost	n_estimators=400, max_depth=8, learning_rate=0.1, subsample=0.9
LightGBM	n_estimators=600, num_leaves=127, learning_rate=0.05, min_child_samples=1
CatBoost	iterations=400, depth=8, learning_rate=0.05
MLP	hidden=(256,128,64), alpha=1e-4, learning_rate_init=0.001
Logistic Regression	C=1.0, tol=1e-3

จุดที่ควรเขียนในรายงาน: LightGBM ตั้ง min_child_samples=1 และปิดการสุ่ม bagging ทั้งสองอย่างตั้งใจไว้ เพราะ Heartbleed มีแค่ 7-8 แถวใน train (1-2 แถวต่อ fold) ถ้าใช้ค่า default คือ 20 โมเดลจะสร้าง leaf ให้คลาสนี้ไม่ได้ และ CV เคยแกว่งเป็นสองกลุ่ม (bimodal: 0.18, 0.09, 0.99, 0.34, 0.99) ตามที่คอมเมนต์ในโค้ดบันทึกไว้